Python

Basic String

using placeholder

```
x = 10
s = "Hello world %d"%x
```

using formatting

```
x = 12
s = f"Hello world {x}"
```

Logging

```
import logging
import numpy as np

logging.basicConfig(filename='logfile.log',
                            format='%(asctime)s %(levelname)s: %(message)s',
                            filemode='w')

logger = logging.setLogger()
logger.setLevel(logging.DEBUG)
x=np.random.random((2,3,4))
logger.debug(f'x :{x}')
logger.info("Just an information")
logger.warning("Its a warning")
logger.error("Did you try to divide by zero?")
logger.critical("Internet is down")
```

Creating a matrix

```
mat1 = [[0]*n_cols]*n_rows
# or
mat2 = [[0]*n_cols for _ in range(n_rows)]
```

Numpy

`np.random.rand(2,3,4)` -> standard uniform

`np.random.random((2,3))` -> random_sample from standard uniform

`np.random.uniform(0,5,(2,3))` -> non-standard uniform distribution

`np.random.randn(2,3,4)` -> standard normal

`np.random.normal(0,5,(2,3))` -> non-standard normal distribution

```
$a1 = np.random.rand(5,10,2)
$b1 = np.random.rand(5,5,2)
$np.hstack((a1,b1)).shape
(5,15,2)
$np.concatenate((a1,b1),axis=1)
a2 = np.random.rand(10,5,2)
b2 = np.random.rand(5,5,2)
$np.vstack((a2,b2)).shape
(15,5,2)
$np.concatenate((a2,b2),axis=0).shape
(15,5,2)
```

`np.nonzero(J)` -> returns tuples of indices of J that are nonzero

`np.maximum(x1, x2)` -> returns he maximum of *x1* and *x2*, element-wise. This is a scalar if both *x1* and *x2* are scalars.

`np.max(J, axis=1)` -> Maximum of *a*. If *axis* is None, the result is a scalar value. If *axis* is an int, the result is an array of dimension `a.ndim - 1`. If *axis* is a tuple, the result is an array of dimension `a.ndim - len(axis)`.

`np.where(J>t, J, 0)` -> fills the position where the boolean condition is true with elements of J, 0 otherwise.

Get indices of N maximum values in a numpy array:

```
n=5
arr=np.random.rand(2,3)
x, y = np.unravel_index(np.argsort(arr.ravel(), axis=None)[::-1][:n], arr.shape)
```

Creating a grid using outer product and `np.meshgrid` for grid indices

Code:

```
# Create a grid-like structure using outer product.
# Used in the implementation of positional encoding
# in Transformers.
a = np.arange(5)
b = np.arange(5)
c = np.arange(5, 0, -1)

grid1 = a[:, np.newaxis] @ b[np.newaxis, :]
grid2 = a[:, np.newaxis] @ c[np.newaxis, :]
print(grid1)
print(grid2)

x, y = np.meshgrid(a, b, indexing='ij')
for (i, j) in zip(x, y):
    print("i: ", i, ", j: ", j)
```

Output:

```
[[ 0  0  0  0  0]
 [ 0  1  2  3  4]
 [ 0  2  4  6  8]
 [ 0  3  6  9 12]
 [ 0  4  8 12 16]]
[[ 0  0  0  0  0]
 [ 5  4  3  2  1]
 [10  8  6  4  2]
 [15 12  9  6  3]
 [20 16 12  8  4]]
i:  [0 0 0 0 0] , j:  [0 1 2 3 4]
i:  [1 1 1 1 1] , j:  [0 1 2 3 4]
i:  [2 2 2 2 2] , j:  [0 1 2 3 4]
i:  [3 3 3 3 3] , j:  [0 1 2 3 4]
i:  [4 4 4 4 4] , j:  [0 1 2 3 4]
```

Indexing in Numpy

Code:

```
import numpy as np


x = np.random.randint(0, 10, (3, 4, 5))
print(x.shape)
print(x)
print('\n')
# l = [i for i in x]
print([i for i in x])  # List comprehension
print('\n')
# The order returned in list comprehension is
# consistent with the order of priority of indexing
# of arrays in numpy.
print(x[0, :])
print(x[0, :, :])
print('\n')
print(x[1, :])
print(x[1, :, :])
print('\n')
print(x[2, :])
print(x[2, :, :])
print('\n')
print((x[0, :] == x[0, :, :]).all())
print((x[1, :] == x[1, :, :]).all())
print((x[1, :] == x[1, :, :]).all())
```

Output:

```
(3, 4, 5)
[[[3 9 8 3 5]
  [4 3 9 9 2]
  [2 5 0 5 6]
  [6 7 1 8 8]]

 [[9 1 2 8 1]
  [8 5 6 8 6]
  [1 3 7 2 9]
  [9 6 3 6 1]]

 [[0 5 7 4 2]
  [2 9 8 1 4]
  [9 0 9 9 8]
  [5 2 5 4 6]]]
```

```
[array([[3, 9, 8, 3, 5],
       [4, 3, 9, 9, 2],
       [2, 5, 0, 5, 6],
       [6, 7, 1, 8, 8]]), array([[9, 1, 2, 8, 1],
       [8, 5, 6, 8, 6],
       [1, 3, 7, 2, 9],
       [9, 6, 3, 6, 1]]), array([[0, 5, 7, 4, 2],
       [2, 9, 8, 1, 4],
       [9, 0, 9, 9, 8],
       [5, 2, 5, 4, 6]])]


[[3 9 8 3 5]
 [4 3 9 9 2]
 [2 5 0 5 6]
 [6 7 1 8 8]]
[[3 9 8 3 5]
 [4 3 9 9 2]
 [2 5 0 5 6]
 [6 7 1 8 8]]


[[9 1 2 8 1]
 [8 5 6 8 6]
 [1 3 7 2 9]
 [9 6 3 6 1]]
[[9 1 2 8 1]
 [8 5 6 8 6]
 [1 3 7 2 9]
 [9 6 3 6 1]]


[[0 5 7 4 2]
 [2 9 8 1 4]
 [9 0 9 9 8]
 [5 2 5 4 6]]
[[0 5 7 4 2]
 [2 9 8 1 4]
 [9 0 9 9 8]
 [5 2 5 4 6]]


True
True
True
```

Matplotlib

1. `fig.add_subplot`

```
# Plot the images in the batch, along with the corresponding labels
fig = plt.figure(figsize=(10, 4))
# Display 20 images
for idx in np.arange(27):
    ax = fig.add_subplot(3, 9, idx + 1, xticks=[], yticks=[])
    imshow(example_data[idx].detach().numpy())
    ax.set_title(classes[example_targets[idx]])
plt.tight_layout()
plt.show()
```

2. `plt.subplots`

```
# Plot the images in the batch, along withe the corresponding labels
fig, subs = plt.subplots(2, 10, figsize=(25, 4))
for idx, sub in zip(np.arange(20), subs.flatten()):
    sub.imshow(np.squeeze(images[idx]), cmap="gray")
    sub.set_title(str(labels[idx].item()))
    sub.axis("off")
plt.savefig("MNIST-data.png")
plt.show()
```